

Пермский институт (филиал)
федерального государственного бюджетного
образовательного учреждения высшего образования
«Российский экономический университет имени Г.В.
Плеханова»
Кафедра информационных технологий и
программирования

ПРАКТИЧЕСКАЯ РАБОТА №1

Тема: Создание и запуск модульных тестов для
управляемого кода

Выполнил: Исмаилов Даниил Эльмардович

Группа: ИПс-11

Преподаватель: Берестов Дмитрий Борисович

Пермь, 2026

Оглавление

Введение	3
Создание основного проекта	3
Создание тестового проекта	6
Создание тестового класса	8
Исправляем код	9
Вывод	12

Введение

В рамках практической работы №1 была изучена работа со средствами модульного тестирования в среде разработки Microsoft Visual Studio 2022. Целью работы являлось создание консольного приложения и разработка для него модульных тестов.

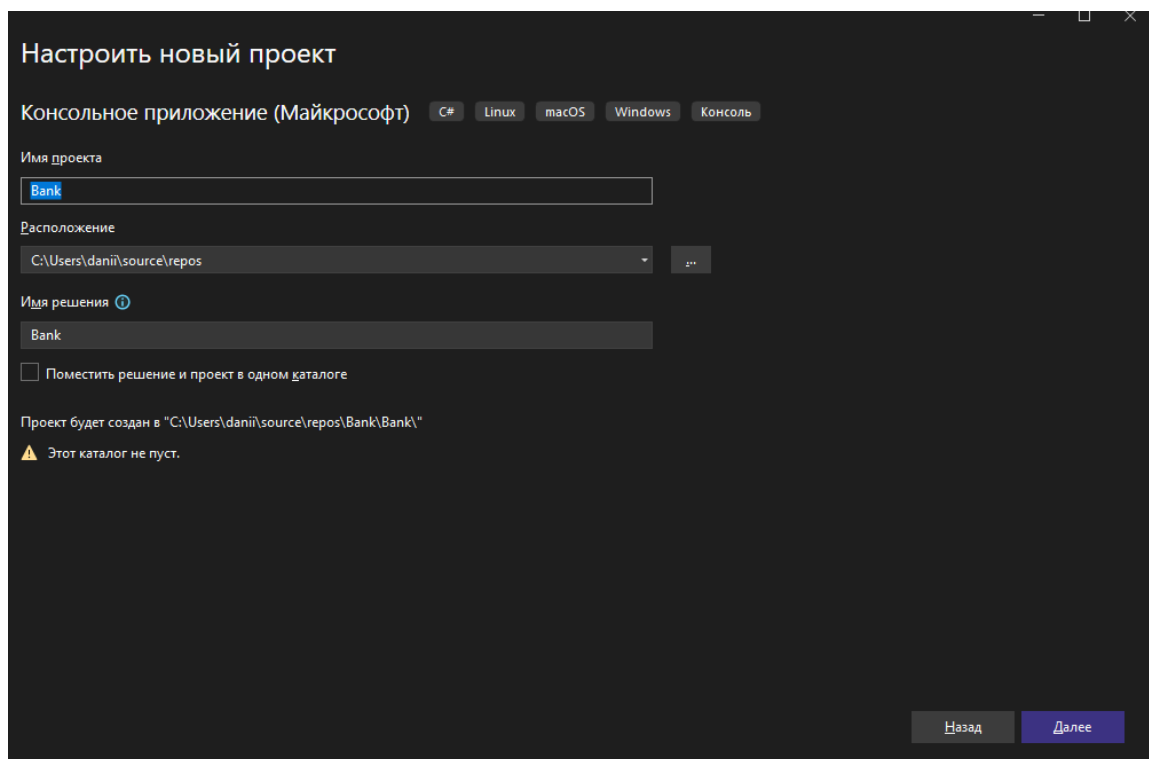
Практическая работа №1 была выполнена на основе предоставленной информации "Средства тестирования Visual Studio-2022", по стр. 158 -170. (<https://cloud.mail.ru/public/JaXA/BUKbRzZoN>).

Создание основного проекта

В среде Visual Studio был создан проект типа «Консольное приложение (.NET Framework)». В проекте реализован класс банковского счета, содержащий поля для хранения имени клиента и баланса, а также методы Debit и Credit для управления денежными средствами.

Создание:

1. Заходим в VS
2. Создаем проект
3. Выбираем из шаблона “Консольное приложение (.NET Framework)”
4. Называем проект “Bank”



5.Заменяем содержимое Program.cs на код который приложен в практической.

```
using System;
namespace BankAccountNS
{
    /// <summary>
    /// Bank account demo class.
    /// </summary>
    public class BankAccount
    {
        private readonly string m_customerName;
        private double m_balance;

        private BankAccount () { }
        public BankAccount(string customerName, double balance)
        {
            m_customerName = customerName;
            m_balance = balance;
        }
        public string CustomerName
        {
            get { return m_customerName; }
        }
        public double Balance
        {
            get { return m_balance; }
        }

        public void Debit(double amount)
        {
            if (amount > m_balance)
            {
                throw new System.ArgumentOutOfRangeException("amount", amount,
                    DebitAmountExceedsBalanceMessage);
            }
            if (amount < 0)
            {
                throw new System.ArgumentOutOfRangeException("amount", amount,
                    DebitAmountLessThanZeroMessage);
            }

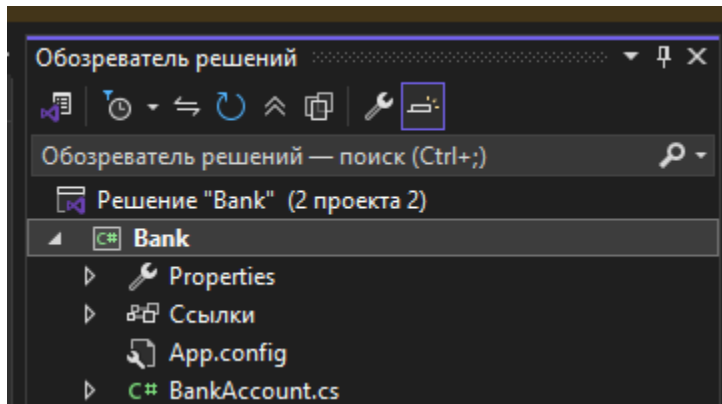
            m_balance -= amount; // intentionally incorrect code
        }
        public void Credit(double amount)
        {
            if (amount < 0)
```

```

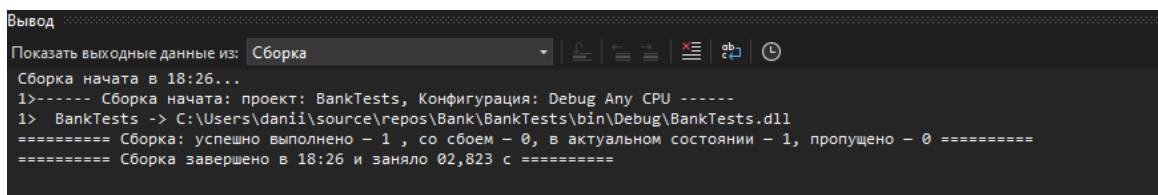
    {
        throw new ArgumentOutOfRangeException("amount");
    }
    m_balance += amount;
}
public static void Main()
{
    BankAccount ba = new BankAccount ("Mr. Bryan Walton",
    11.99);
    ba.Credit(5.77);
    ba.Debit(11.22);
    Console.WriteLine("Current balance is ${0}", ba.Balance);
}
}
}

```

6. В обозревателе решений переименовываем Program.cs на BankAccount.cs



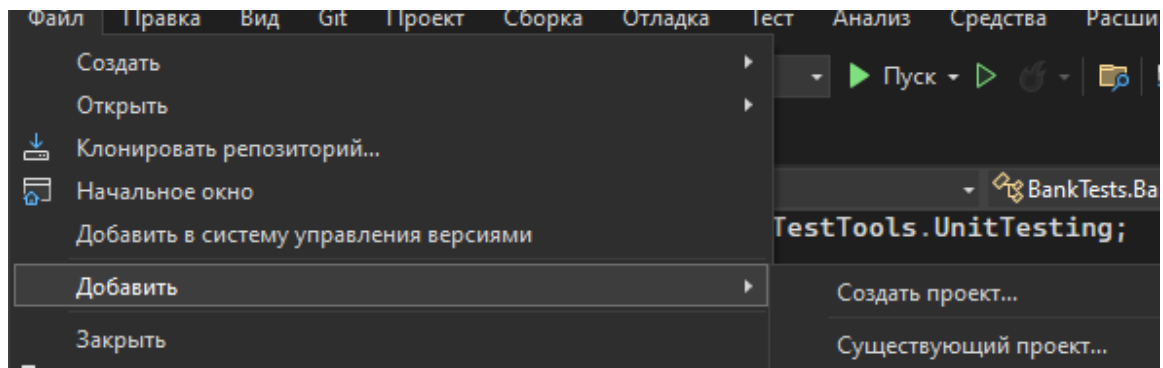
7. Теперь нажимаем сочетание клавиш CTRL+SHIFT+B, так мы построим решение



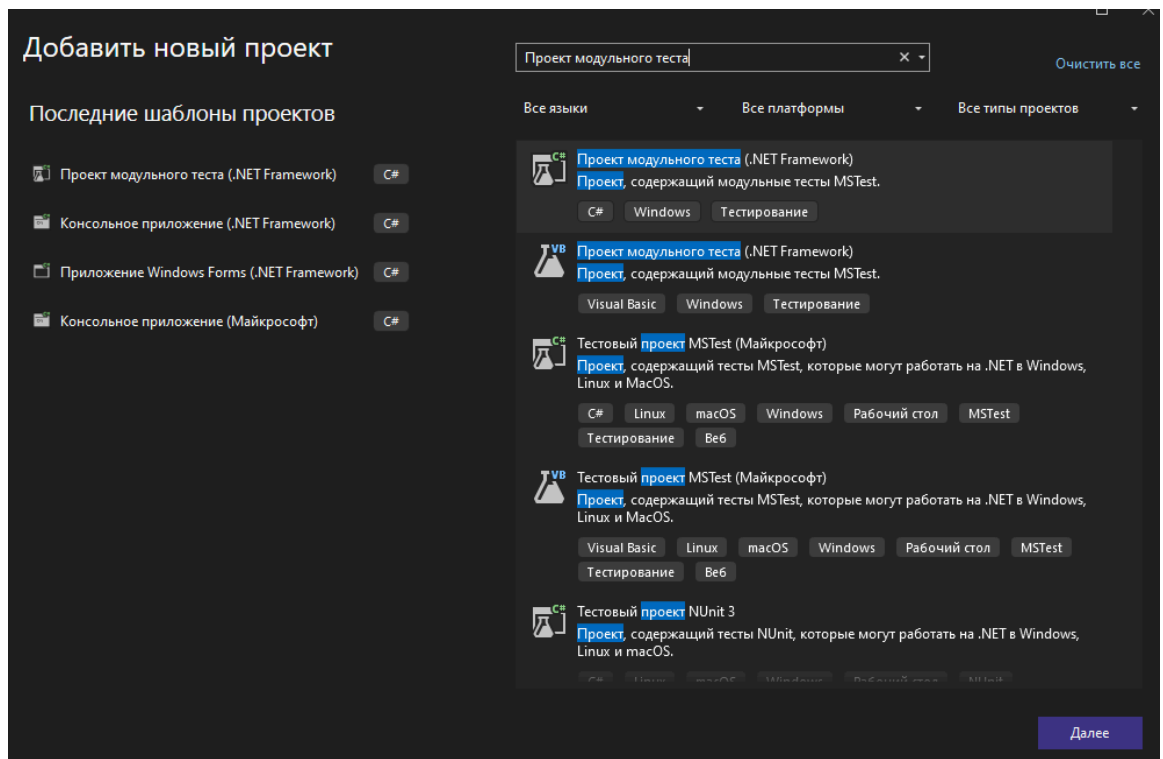
Создание тестового проекта

Далее создадим проект «Проект модульного тестирования (.NET Framework)», который добавим в текущее решение. После этого настроим ссылку на основной проект.

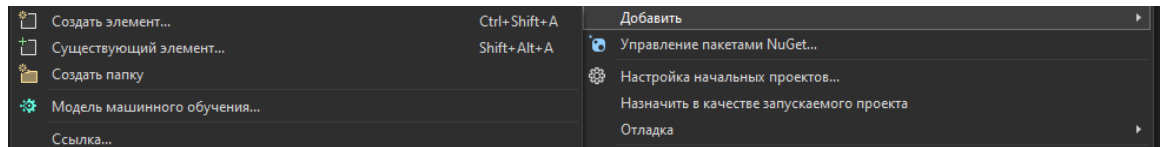
- 1) Слева сверху в меню файл нажмем Добавить - создать проект



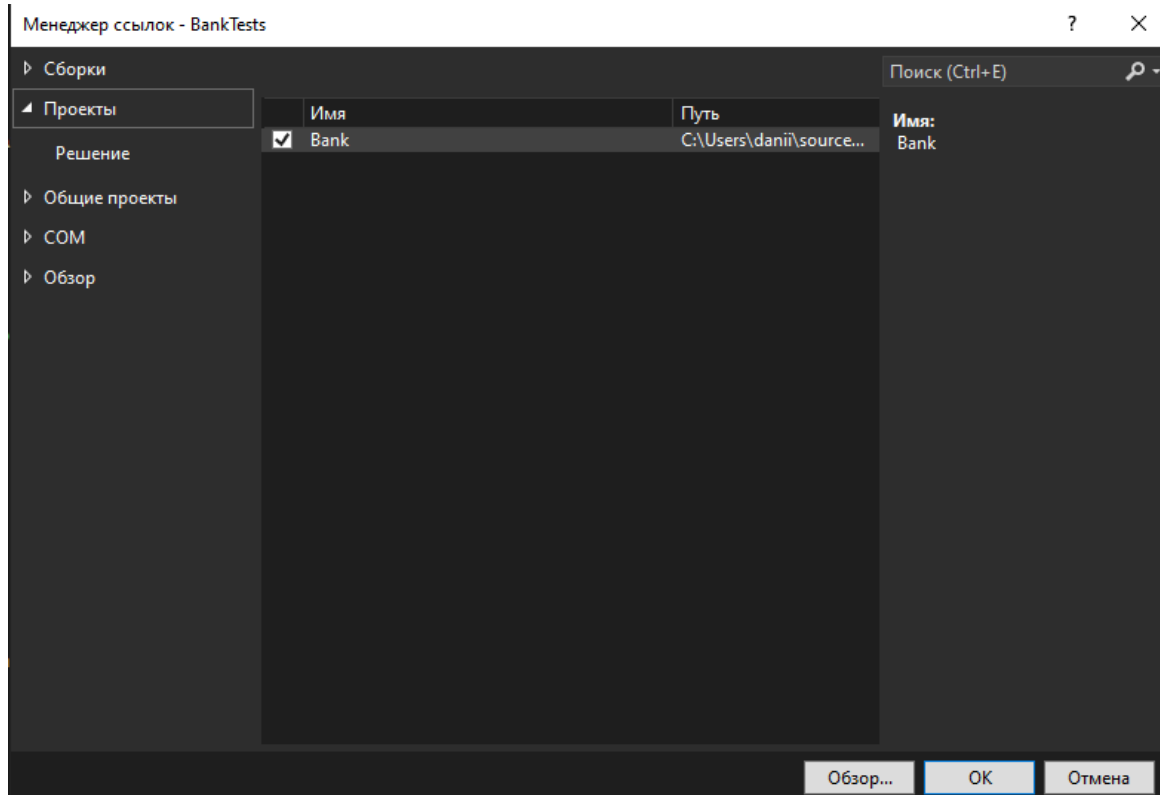
- 2) В поиске шаблона вписываем «Проект модульного теста MSTest» и выбираем «Проект модульного теста (.NET Framework)»



- 3) Назовите проект BankTests и щелкните Далее
- 4) Добавим ссылку через обозреватель решений



- 5) Ставим галочку и нажимаем ОК



Создание тестового класса

1. В "BankTests" переименовываем "UnitTest1.cs" на "BankAccountTest.cs"
2. Добавляем оператор using BankAccountNS;

```

using Microsoft.VisualStudio.TestTools.UnitTesting;
using BankAccountNS;
using System;

namespace BankTests
{
    [TestClass]
    Ссылка: 0
    public class BankAccountTests
    {
        [TestMethod]

```

Далее создаем первый метод теста

```

[TestMethod]
Ссылка: 0
public void Debit_WithValidAmount_UpdatesBalance()
{
    // Arrange
    double beginningBalance = 11.99;
    double debitAmount = 4.55;
    double expected = 7.44;

    BankAccount account = new BankAccount("Mr. Bryan Walton", beginningBalance);

    // Act
    account.Debit(debitAmount);

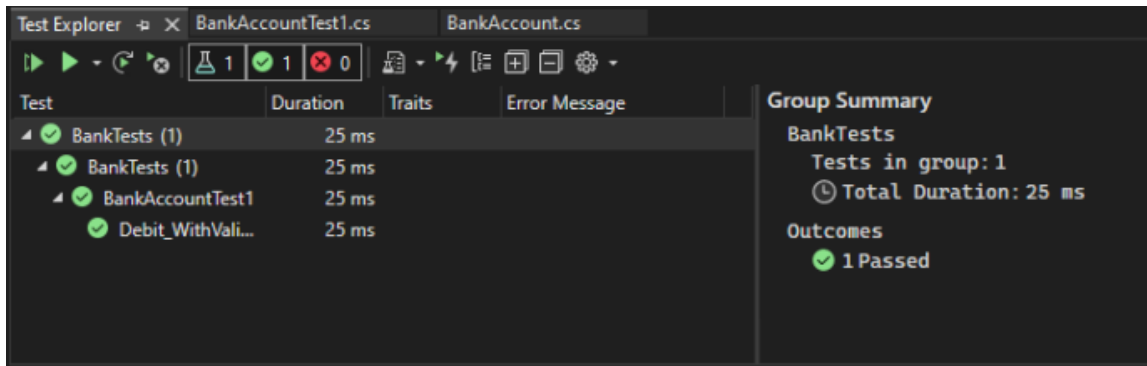
    // Assert
    double actual = account.Balance;
    Assert.AreEqual(expected, actual, 0.001, "Account not debited correctly");
}

```

Нажимаем CTRL+SHIFT+B и видим ошибку.

Исправляем код

В процессе тестирования была выявлена ошибка в логике изменения баланса. исправляем - m_balance += amount; (Меняем "+" на "-")



Как видите, все работает.

Продолжаем анализ и добавляем новые методы тестирования, все должно выглядеть так:

Первый метод:

```
using Microsoft.VisualStudio.TestTools.UnitTesting;
using BankAccountNS;
using System;

namespace BankTests
{
    [TestClass]
    Ссылка: 0
    public class BankAccountTests
    {
        [TestMethod]
        Ссылка: 0
        public void Debit_WithValidAmount_UpdatesBalance()
        {
            // Arrange
            double beginningBalance = 11.99;
            double debitAmount = 4.55;
            double expected = 7.44;

            BankAccount account = new BankAccount("Mr. Bryan Walton", beginningBalance);

            // Act
            account.Debit(debitAmount);

            // Assert
            double actual = account.Balance;
            Assert.AreEqual(expected, actual, 0.001, "Account not debited correctly");
        }
    }
}
```

Второй метод:

```

[TestMethod]
Ссылка: 0
public void Debit_WhenAmountIsLessThanZero_ShouldThrowArgumentOutOfRangeException()
{
    // Arrange
    double beginningBalance = 11.99;
    double debitAmount = -100.00;

    BankAccount account = new BankAccount("Mr. Bryan Walton", beginningBalance);

    // Act
    try
    {
        account.Debit(debitAmount);

        Assert.Fail("Expected ArgumentOutOfRangeException was not thrown.");
    }
    catch (ArgumentOutOfRangeException e)
    {
        // Assert
        StringAssert.Contains(
            e.Message,
            BankAccount.DebitAmountLessThanZeroMessage);
    }
}

```

Третий метод:

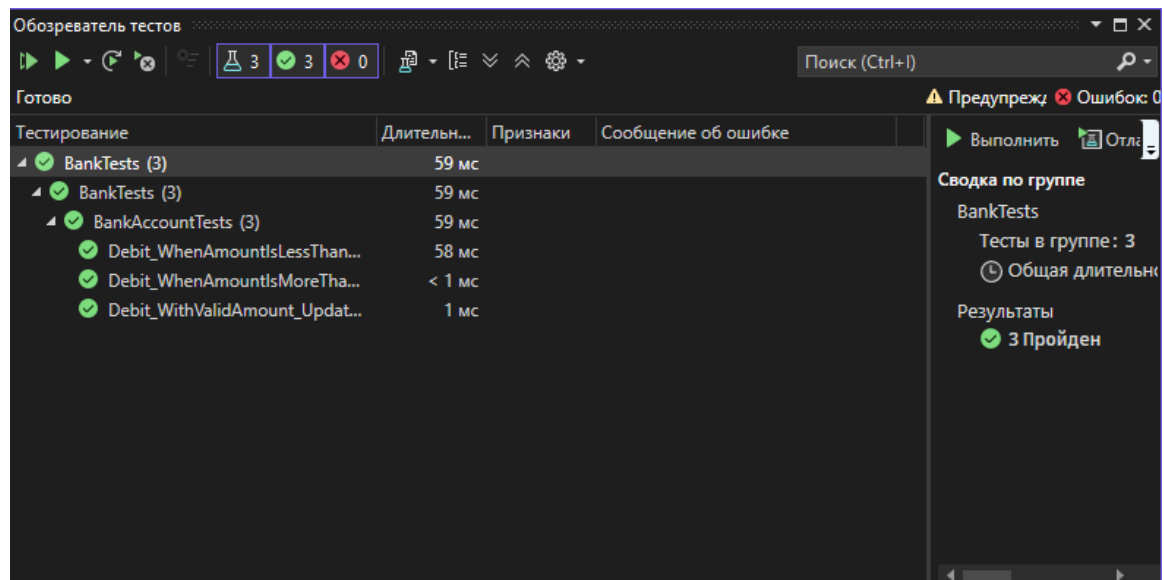
```

[TestMethod]
Ссылка: 0
public void Debit_WhenAmountIsMoreThanBalance_ShouldThrowArgumentOutOfRangeException()
{
    // Arrange
    double beginningBalance = 11.99;
    double debitAmount = 20.0;
    BankAccount account = new BankAccount("Mr. Bryan Walton",
beginningBalance);

    // Act
    try
    {
        account.Debit(debitAmount);
    }
    catch (System.ArgumentOutOfRangeException e)
    {
        // Assert
        StringAssert.Contains(e.Message,
BankAccount.DebitAmountExceedsBalanceMessage);
        return;
    }
    Assert.Fail("The expected exception was not thrown.");
}
}

```

Теперь нажимаем сверху Тест - Запуск всех тестов



Как видим все работает

Вывод

В результате выполнения практической работы я освоил базовые навыки создания и запуска модульных тестов в Visual Studio. Работа позволила мне лучше понять важность тестирования программного кода и обработки исключительных ситуаций. Спасибо за новый опыт.